

Bottom-Up-Parser

⇒ In bottom up parsing, the parser constructs the parse tree, starting with the input string and moves towards the root or start symbol of grammar by tracing out the rightmost derivative in reverse.

OR

⇒ Bottom up parser tries to reduce the input string w to the start symbol of grammar by tracing out the rightmost derivatives of w in reverse.

⇒ If the given string is reduced to the start symbol then its syntax is valid.

⇒ Otherwise bottom-up parser will generate an error message.

⇒ The parse tree is constructed from bottom to up that is from leaves to root.

⇒ The bottom up parse tree is created starting from leaves. The leaf nodes together are reduced further to internal nodes, these internal nodes are further reduced and eventually a root node is obtained.

⇒ The internal nodes are created from the list of terminal and non-terminal symbols.

This involves -

∴ Non-terminal for internal node =
Nonterminal $\cup$ terminal.

⇒ In this process, basically parser tries to identify R.H.S of production rule and replace it by corresponding L.H.S. This activity is called reduction.

⇒ The crucial but prime task in bottom-up parsing is to find the productions that can be used for reduction.

⇒ The bottom-up parse tree construction process indicates that the tracing of derivations are to be done in reverse order.

For example:

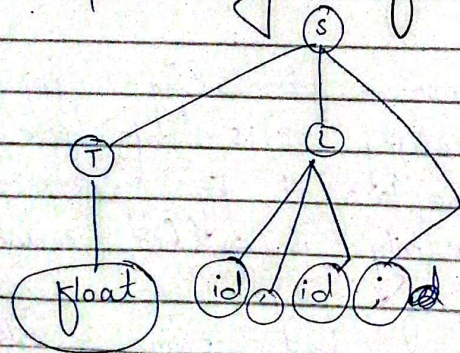
Consider the grammar

$S \rightarrow TL;$

$T \rightarrow \text{int} \mid \text{float}$

$L \rightarrow L, \text{id} \mid \text{id}$

The input string is float id, id;



⇒ it is also known as Shift-reduce parser.

↑ Problem of discarding a handle and reducing it to the appropriate left hand side is called handle pruning.

Handle Pruning:

⇒ The crucial task in bottom-up parsing is to find the substring that could be reduced by appropriate non-terminal. Such a substring is called handle.

⇒ In other words, handle is a string of substring that matches the right side of production and we can reduce such string by a non-terminal on left hand side production.

Definition of Handle: → substring that matches the R.H.S of the production & whose reduction represents one step along the reverse of a RMP.

o " Handle of right sentential form γ is a production $A \rightarrow \beta$ and a position of γ where the string β may be found and replaced by A to produce the previous right sentential form in rightmost derivation of γ ."

for example: primitive algebra
 Consider the grammar:

$$E \rightarrow E + E$$

$$E \rightarrow id$$

Now, Consider the String:

id+id+id

and the right most derivation is.

$$E \Rightarrow_{\delta m} E + E$$

$$E \Rightarrow_{\delta m} E + E + E$$

$$E \Rightarrow_{\delta m} E + (E + id)$$

$$E \Rightarrow_{\delta m} E + id + id$$

$$E \Rightarrow_{\delta m} id + id + id$$

The bold strings are called handle.

Right Sentential form	Handle	Production
id+id+id	id	$E \rightarrow id$
E+id+id	id	$E \rightarrow id$
E+E+id	id	$E \rightarrow id$
E+E+E	E+E	$E \rightarrow E+E$
E+E	E+E	$E \rightarrow E+E$
E		

$\Rightarrow$ Thus bottom-up parser is actually a process of detecting handles and using them in production. This process is called handle pruning.

Shift-reduce parser

$\Rightarrow$ Shift reduce parser attempts to construct parse tree from leaves to root. Thus, it works on the same principle of bottom up parser.

$\Rightarrow$ Data-Structures used in Shift-reduce parser are:

- 1) The input buffer storing the input string.
- 2) A stack for storing and accessing the L.H.S and R.H.S of rules.

Initial Configuration of Shift-reduced parser:

Stack
 \$S

i/p buffer
 W\$

Tries to reduce the i/p string to the start symbol of the grammar by using RMD in reverse.

Basic Operations:-

The parser performs following basic operations:

1. Shift:

• Moving of the symbols from input buffer onto the stack, this action is called shift.

2. Reduce: (symbol replaced by non-terminal).
 • If the handle appears on the top of the stack then reduction of it done by appropriate rule is done.
 • That means R.H.S of rule is popped off and L.H.S is pushed in. This action is called reduction.

3. Accept:

• If the stack contains start symbol only and the i/p buffer is empty at the same time then that action is called accept. When accept state is obtained in the process of parsing then it means a successful parsing is done.

4. Error:

• A situation in which parser cannot either shift or reduce the symbols, it cannot even perform the action is called as error.

Example:

$$E \rightarrow E - E$$

$$E \rightarrow E * E$$

$$E \rightarrow id$$

String = $id - id * id$
 'high precedence.

Stack	I/P Buffer	Parser Action
\$	$id_1 - id_2 * id_3 \$$	Shift
$\$ id_1$	$- id_2 * id_3 \$$	Reduce by $E \rightarrow id$
$\$ E$	$- id_2 * id_3 \$$	Shift
$\$ E -$	$id_2 * id_3 \$$	Shift
$\$ E - id_2$	$* id_3 \$$	Reduce by $E \rightarrow id$
$\$ E - E$	$id_3 \$$	Shift
$\$ E - E *$	$\$$	Shift
$\$ E - E * id_3$	$\$$	Reduce by $E \rightarrow E * E$
$\$ E - E * E$	$\$$	Reduce by $E \rightarrow E - E$
$\$ E - E$	$\$$	Reduce by $E \rightarrow E - E$
$\$ E$	$\$$	Accept

Here two rules are followed:-

=> if the incoming operator has more priority than in stack operator then perform shift.

=> if in stack operator has same or less priority than the priority of incoming operator then perform reduce.

Example #2:-

Consider the following grammar

$S \rightarrow TL;$

$T \rightarrow \text{int} | \text{float}$

$L \rightarrow L, \text{id} | \text{id}$

Parse the input string $\text{int id, id};$ using Shift reduce parser.

Stack	i/p buffer	Passing Action
\$	int id, id; \$	Shift
\$int	id, id; \$	Reduce by $T \rightarrow \text{int}$
\$T	id; \$	Shift
\$Tid	id; \$	Shift
\$Tid,	;\$	Shift Reduce $L \rightarrow L,$
\$Tid, id	;\$	Reduced by $L \rightarrow L,$
\$TL	;\$	Shift
\$TL;	\$	Reduced by $S \rightarrow TL;$

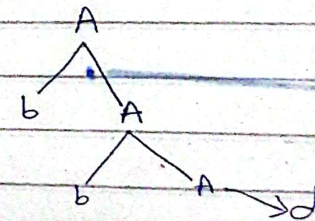
Q How top down and bottom up passes will parse the string 'b b d' using grammar $A \rightarrow b A | d$

For top down passing LL(1) passing method is used

	FIRST	FOLLOW
$A \rightarrow b A d$	{b, d}	{ \$ }

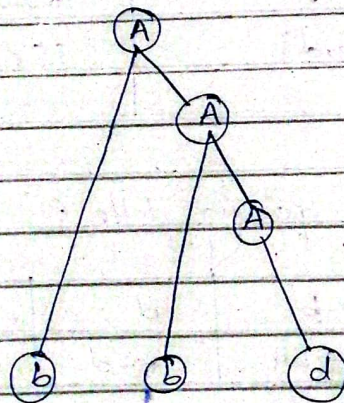
The predictive parsing table:

	b	d	\$
A	$A \rightarrow b A$	$A \rightarrow d$	



o for bottom up parsing we will use
Shift reduce parsing method.

Stack	i/p buffer	Parsing Action
\$	\$	Shift
\$b	bd	Shift
\$bd	d	Shift
\$b b d	\$	Reduced by $A \rightarrow bA$
\$b b A	\$	Reduced by $A \rightarrow bA$
\$ b A	\$	Reduced by $A \rightarrow bA$
\$ A	\$	Accept



Operator precedence parser:-

Operator precedence grammar:

A grammar that satisfies the following 2 conditions is called operator precedence grammar:

- (1) There exists no production rule which contains ϵ on its R.H.S.
- (2) There exists no production rule which contains two non-terminals adjacent to each other on its R.H.S.

⇒ It represents a small class

of grammars.

But it is important class because of its widespread application.

"A grammar that is used to define mathematical operators is called an operator precedence grammar or operator grammar."

Consider the grammar for arithmetic expressions.

$$E \rightarrow EAE \mid (E) \mid -E \mid id$$

$$A \rightarrow + \mid - \mid * \mid / \mid \wedge$$

⇒ This grammar is not operator precedence grammar as the production rule

$$E \rightarrow EAE$$

it contains two consecutive non-terminals

Ex:

$$E \rightarrow E+E \mid E-E \mid E * E \mid E / E \mid E \wedge E \mid (E) \mid -E \mid id$$

Operator precedence Parser:

⇒ "A parser that reads and understands an operator precedence grammar is called as operator precedence parser."

⇒ This parser is only used for operator grammar

Note:

⇒ Ambiguous grammars are not allowed in any parser except operator precedence grammars.

Designing Operator Precedence Parser:-

⇒ In operator precedence parsing,

o Firstly, we define precedence relations between every pair of terminal symbols.

o Secondly, we construct an operator precedence table.

Designing Precedence Relations:

⇒ The precedence relations are defined using the following rules:-

Rule-01:

- o if precedence of b is higher than precedence of a , then we define $a < b$.
- o if precedence of b is same as precedence of a , then we define $a = b$.
- o if precedence of b is lower than precedence of a , then we define $a > b$.

Rule-02:-

- o An identifier is always given the highest precedence than any other symbol.
- o $\$$ symbol is always given the lowest precedence.

Rule-03:-

- o if two operators have the same precedence, then we go by checking their associativity.

Operator precedence table

	+	-	*	/	^	id	(	)	\$
+	>	>	<	<	<	<	<	>	>
-	>	>	<	<	<	<	<	>	>
*	>	>	>	>	<	<	<	>	>
/	>	>	>	>	<	<	<	>	>
↑	>	>	>	>	<	<	<	>	>
id	>	>	>	>	>	X	X	>	>
id	>	>	>	>	>	X	X	>	>
id	<	<	<	<	<	X	X	<	<
\$	<	<	<	<	<	<	<	<	<

Notes

Operator precedence table is not stored by the Operator precedence parser. This is because it occupies the large space.

o Instead, Operator precedence parsers are implemented in a very unique style.

o They are implemented using operator precedence functions.

Precedence parsing

- o This type of parsing is simple to implement.
- o The parser is quite powerful for expressions in programming languages.

Cons:-

- o The operator like minus has two different precedence (unary and binary). The handling of these tokens becomes difficult.
- o Only small class of grammars can be parsed using this parser.

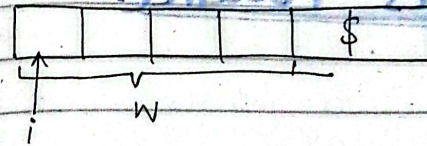
Operator precedence functions:

- o Precedence functions perform the mapping of terminal symbols to the integers.

→ To decide the precedence relation b/w symbols, a numerical comparison is performed.

Operator precedence Parsing Algorithm:-

- o Set i pointer to first symbol of string w . The string will be represented as follows:



- o if $\$$ is the top of the stack and if a is the symbol pointed by i then return.
- o if a is on the top of the stack and if the symbol b is read by pointer i then
 - a) if $a < b$ or $a = b$ then push b on the stack advance the pointer i to next i/p symbol.
 - b) else if $a > b$ then while (top symbol of the stack $\rightarrow$ recently popped terminal symbol)

the symbol means reducing the symbol by equivalent non-terminal

c) else

error()

Examples Practice

Q. Consider the following grammar

$$E \rightarrow EAE | id$$

$$A \rightarrow + | x$$

Construct the operator precedence parser and parse the string $id + id * id$

o The equivalent operator precedence grammar:

$$E \rightarrow E + E | E * E | id$$

o The terminal symbols are $id, +, x, \$$.

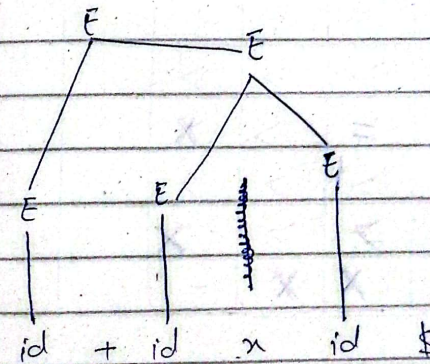
Now construct the precedence table.

	id	+	x	\$
id	-	>	>	>
+	<	>	<	>
x	<	>	>	>
\$	<	<	<	-

id + id x id \$

Stack: \$ id + id x id

Pop Pop id > x
Pop id



EX2: Consider the following grammar

$$S \rightarrow (L) | a$$

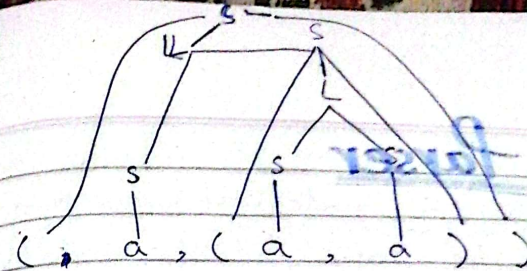
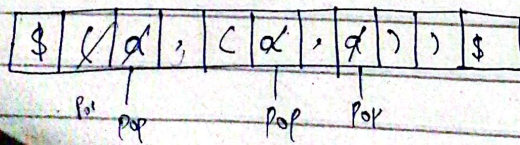
$$L \rightarrow L, S | S$$

Construct the operator precedence parser and parse tree $(a, (a, a))$.

The terminal symbols in the grammar are $\{ (,), a, , \}$.

Now, Construct the operator precedence table:-

	a	(	)	,	\$
a	-	X	>	>	>
(	<	>	=	>	X
)	X	X	>	>	>
,	<	<	X	>	X
\$	<	<	X	X	-



Precedence of (,), , is